Long Hair Gender Identification System

A Multi-Model Approach Combining Age Estimation, Gender Classification and Hair Length Detection

Machine Learning Project Report

Built with TensorFlow, PyTorch and Streamlit

Contents

1. Problem Statement

The task is to build a gender identification feature with an unusual twist in its logic. Instead of simply predicting a person's true gender, the system has to follow a specific rule that depends on the person's age and hair length.

For people aged between 20 and 30: the system should classify anyone with long hair as female, even if they are actually male, and anyone with short hair as male, even if they are actually female. In this age band, hair length overrides true gender.

For people outside this range (under 20 or over 30): the system should ignore hair length completely and simply predict the person's real gender.

The task explicitly asks for a self-built machine learning model and a working graphical user interface. Evaluation is based on the overall behaviour of the model and the functionality of the interface, not purely on raw accuracy. This makes it as much a logic-building exercise as a modelling one, because the core challenge is figuring out which sub-problems need solving and how to wire them together correctly.

2. Understanding the Problem

On the surface this looks like a gender classification problem, but reading it carefully shows it is really three problems stacked on top of each other. To produce the required output for any given face, the system needs to know three independent facts about the person:

- Their approximate age, so it can decide which rule applies (the 20–30 rule or the normal-gender rule).
- Their hair length, which becomes the deciding factor inside the 20–30 band.
- Their actual gender, which is the answer used everywhere outside the 20–30 band.

This means no single model can solve the task. The solution has to be a pipeline: one model predicts age and gender, a second model predicts hair length, and then a small piece of decision logic combines those three predictions according to the rule in the problem statement. Identifying this structure early was the most important step in the whole project, because it turned a confusing single requirement into three clear, solvable sub-tasks.

The final decision rule can be written in plain language as follows:

```
if 20 <= age <= 30:     gender = 'Female' if hair == 'long'
else 'Male' else:
    gender = predicted_real_gender
```

Everything else in this report — the two neural networks, the datasets, the GUI — exists to supply the three values that feed into this rule.

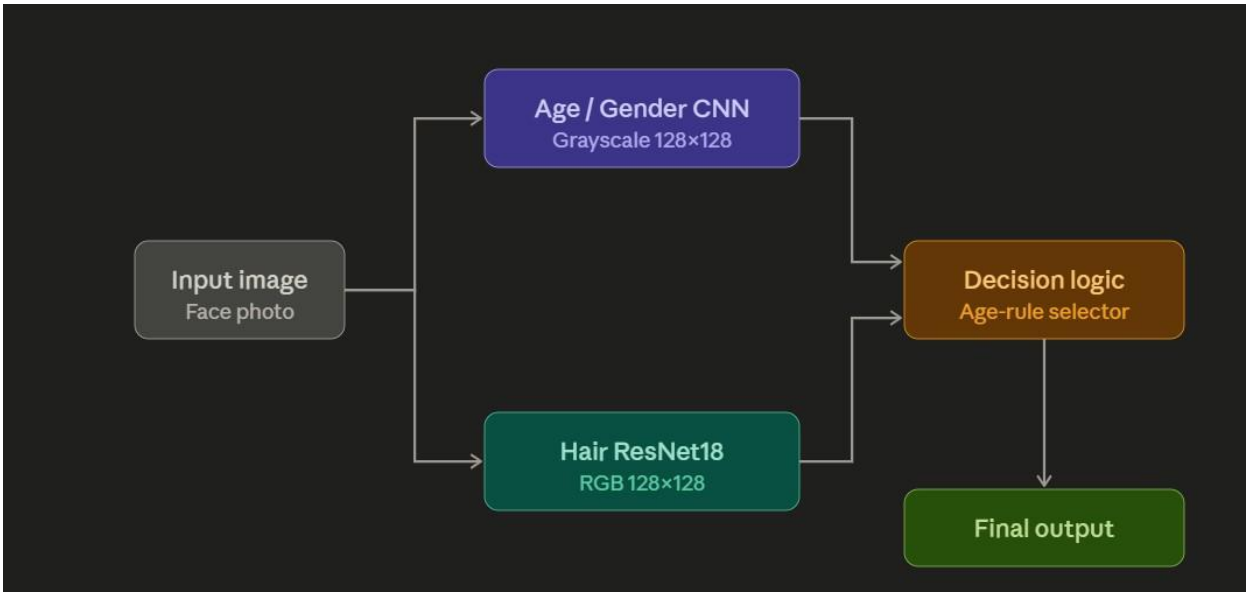
3. System Overview

The system is made up of two trained models and one decision layer, all wrapped inside a single web interface. When a user uploads a photo, the image is sent through both models in parallel and their outputs are merged by the decision logic before anything is shown on screen.

Component	Framework	What it produces
Age + Gender model	TensorFlow / Keras	Estimated age (number) and real gender (male/female)

Hair length model	PyTorch (ResNet18)	Hair length (long/short) with a confidence score
Decision logic	Plain Python	Applies the 20–30 rule and outputs the final gender label
GUI	Streamlit	Lets the user upload an image and view all results

[INSERT A SIMPLE BLOCK DIAGRAM HERE: Image → [Age/Gender CNN] + [Hair ResNet18] → Decision Logic → Final Output. You can draw this in PowerPoint or draw.io]



4. Datasets Used

4.1 UTKFace

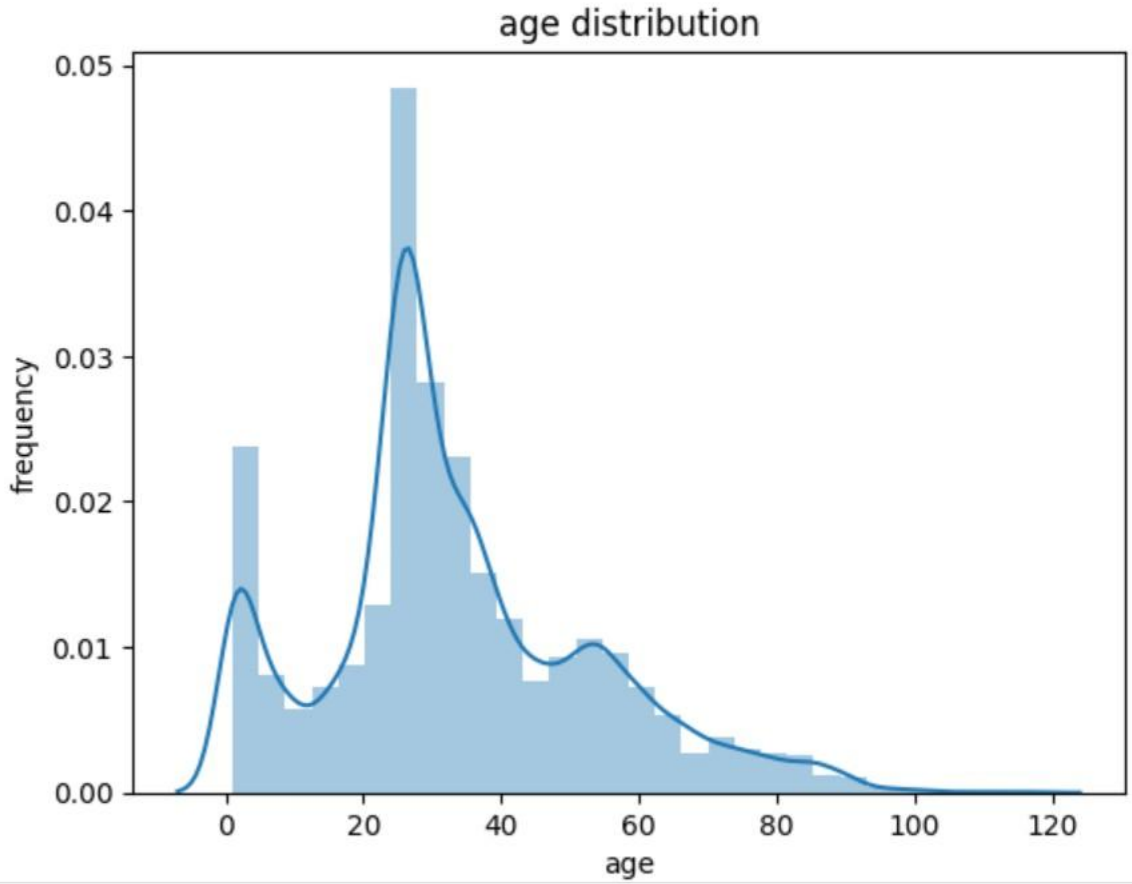
UTKFace is a large public face dataset of more than twenty thousand images. Its most useful feature is that the age, gender and ethnicity of each person are stored directly inside the filename, separated by underscores. For example a file named 25_0_2_xxxx.jpg means a 25-year-old male. This made loading labels extremely simple, because the labels could be parsed straight from the filenames without any separate annotation file.

The code that reads the dataset loops over every file, splits the filename on the underscore character, and pulls out the age and gender from the first two fields:

```
for filename in os.listdir(BASE_DIR):
    temp = filename.split('_')
age    = int(temp[0])     gender
= int(temp[1])
    image_paths.append(os.path.join(BASE_DIR,
filename))     age_labels.append(age)
gender_labels.append(gender)
```

These three lists were then placed into a pandas DataFrame so the data could be inspected and manipulated easily. UTKFace was used both to train the age and gender model and as the image source for building the hair length dataset.

```
Text(0, 0.5, 'frequency')
```



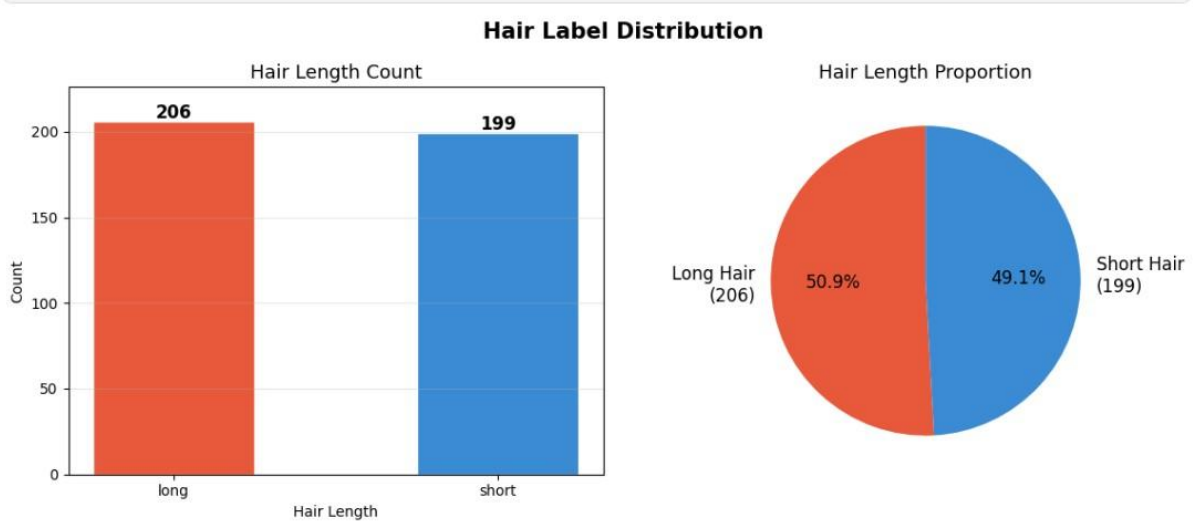
[

4.2 Manually Labelled Hair Dataset

UTKFace does not contain any information about hair length, so this had to be created by hand. Five hundred images were sampled from the 20 to 30 age range, balanced as 250 males and 250 females, and exported from the notebook as a zip file. Each image was then labelled manually as long, short, or skip. The skip option was used for images where hair length genuinely could not be judged, such as people wearing hats or photographed from an angle that hid their hair. After removing the skipped images, around 405 clean labels remained, which were saved to a CSV file and re-uploaded to Kaggle as a private dataset.

Manual labelling specifically targeted the 20 to 30 band because that is the only range where hair length actually matters for the final decision. Spending the labelling effort exactly where the problem needs it was a deliberate choice to get the most value from a small amount of hand-annotation work.

[Bar chart and pie chart of the hair label distribution (long vs short counts from hair_labels.csv)]



✓ Saved as hair_label_distribution.png

5. Task 1 — Age and Gender Model (CNN)

5.1 Why a Convolutional Neural Network

Age and gender are both visual properties of a face, so a Convolutional Neural Network is the natural choice. A CNN works by sliding small filters across the image to detect simple patterns like edges and curves in its early layers, and then combining those into more complex features such as eyes, jaw shape and skin texture in its deeper layers. These are exactly the kinds of features that distinguish a young face from an old one, or a male face from a female one.

5.2 A Single Model with Two Outputs

Rather than building two separate networks, one model was designed with a shared body and two output heads. The convolutional layers are shared, and near the end the network splits into two branches: one that predicts gender and one that predicts age. This is called multi-task learning. The advantage is efficiency: the shared layers learn general facial features that are useful for both tasks at once, which usually trains faster and generalises better than two isolated models.

The model was built using the Keras functional API, which allows a network to have multiple outputs. The structure is four convolutional blocks, each followed by a max-pooling layer that halves the spatial size, then a flatten, then two separate dense branches:

```
inputs = Input((128, 128, 1))

conv1 = Conv2D(32, (3,3), activation='relu')(inputs) maxp1
= MaxPooling2D((2,2))(conv1)
conv2 = Conv2D(64, (3,3), activation='relu')(maxp1) maxp2
= MaxPooling2D((2,2))(conv2)
conv3 = Conv2D(128,(3,3), activation='relu')(maxp2) maxp3
= MaxPooling2D((2,2))(conv3)
conv4 = Conv2D(256,(3,3), activation='relu')(maxp3) maxp4
= MaxPooling2D((2,2))(conv4)

flatten = Flatten()(maxp4)
dense1  = Dense(256, activation='relu')(flatten)   # gender
branch dense2  = Dense(256, activation='relu')(flatten)   # age
branch drop1   = Dropout(0.3)(dense1) drop2   =
Dropout(0.3)(dense2)

gender_out = Dense(1, activation='sigmoid', name='gender_out')(drop1) age_out
= Dense(1, activation='relu',    name='age_out')(drop2)

model = Model(inputs=inputs, outputs=[gender_out, age_out])
```

5.3 Understanding the Key Layers

Conv2D: each convolutional layer applies a set of learnable filters across the image. The number of filters grows through the network (32, 64, 128, 256), letting deeper layers detect increasingly complex and abstract features. The 3x3 refers to the size of each filter window.

MaxPooling2D: after each convolution, max-pooling shrinks the feature map by keeping only the strongest activation in each 2x2 region. This reduces computation, makes the network less sensitive to the exact position of features, and helps prevent overfitting.

ReLU activation: this is the function that introduces non-linearity, allowing the network to learn complex relationships rather than just straight-line mappings. It simply turns any negative value into zero and leaves positive values unchanged.

Flatten: the final pooled feature maps are a 3D block of numbers. Flatten unrolls this into a single long vector so it can be passed into ordinary dense layers.

Dropout (0.3): during training, dropout randomly switches off 30 percent of the neurons in a layer on each step. This forces the network not to rely too heavily on any single neuron, which reduces overfitting on the training data.

The two output layers: the gender output uses a sigmoid activation, which squashes its result into a value between 0 and 1 that can be read as the probability of being female. The age output uses ReLU, which produces a non-negative number representing the predicted age directly as a continuous value.

5.4 Preprocessing the Images

Before training, every image was converted to greyscale, resized to 128 by 128 pixels, and stacked into a single array. Greyscale was chosen on purpose: age and gender are determined mostly by the structure and shape of a face rather than its colour, so dropping colour reduces the input size to one third without losing much useful information. Pixel values were then divided by 255 to scale them into the 0 to 1 range, which helps the network train more stably.

```
def
extract_feature(images):
features = []     for image
in images:
        img = Image.open(image).convert('L')      #
greyscale         img = img.resize((128, 128))
features.append(np.array(img))     features =
np.array(features)
    return features.reshape(len(features), 128, 128, 1)

x = extract_feature(df['Image']) / 255.0
```

5.5 Training Setup

The data was split into 80 percent training, 10 percent validation and 10 percent test. Because the model has two outputs, it was compiled with two separate loss functions: binary crossentropy for the gender output, which is a classification problem, and mean absolute error for the age output, which is a regression problem. The Adam optimiser was used, and a learning rate scheduler gradually reduced the learning rate by ten percent every epoch so the model could take smaller, more careful steps as it converged.

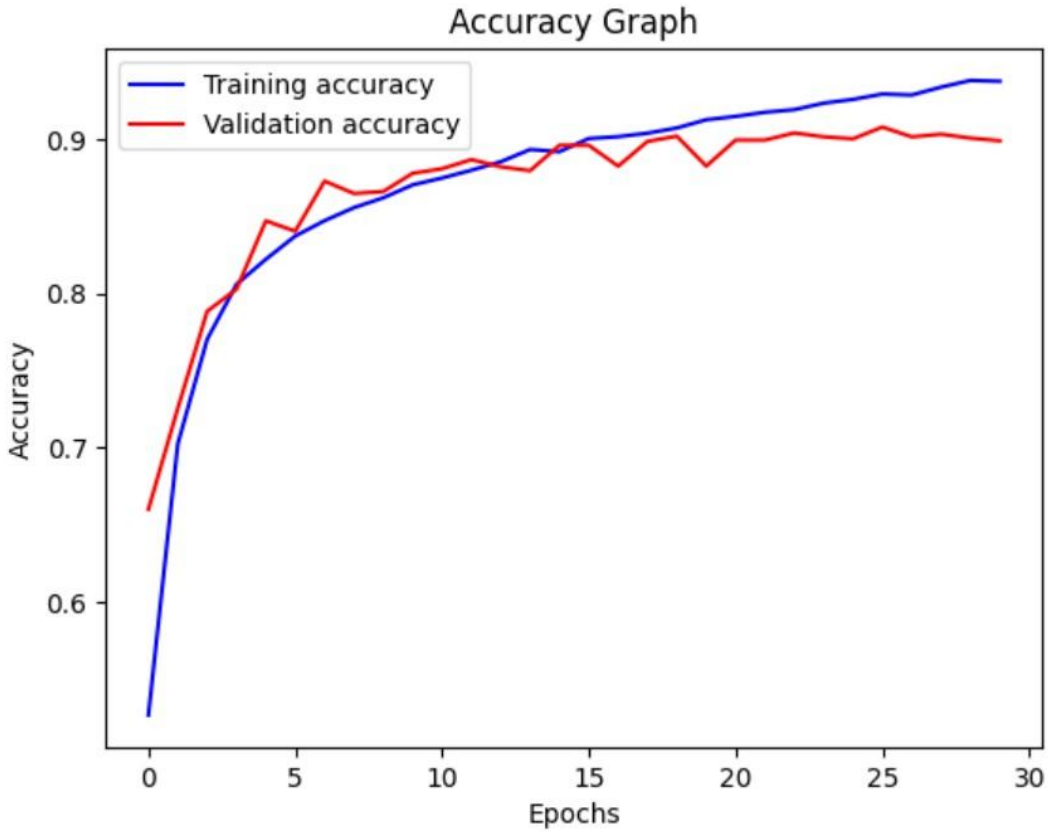
```
model.compile(
    loss=['binary_crossentropy', 'mae'],     optimizer='adam',
    metrics=['accuracy', 'mae']
)

annealer = LearningRateScheduler(lambda e: 1e-3 * 0.9 ** e)

history = model.fit(
    X_train, [y_train_gender, y_train_age],     batch_size=128,
epochs=30,
    validation_data=(X_val, [y_val_gender, y_val_age]),
callbacks=[checkpoint]
)
```

A ModelCheckpoint callback watched the validation gender accuracy and saved only the best-performing version of the model to disk, so that the final saved file represented the model at its peak rather than at the last epoch.

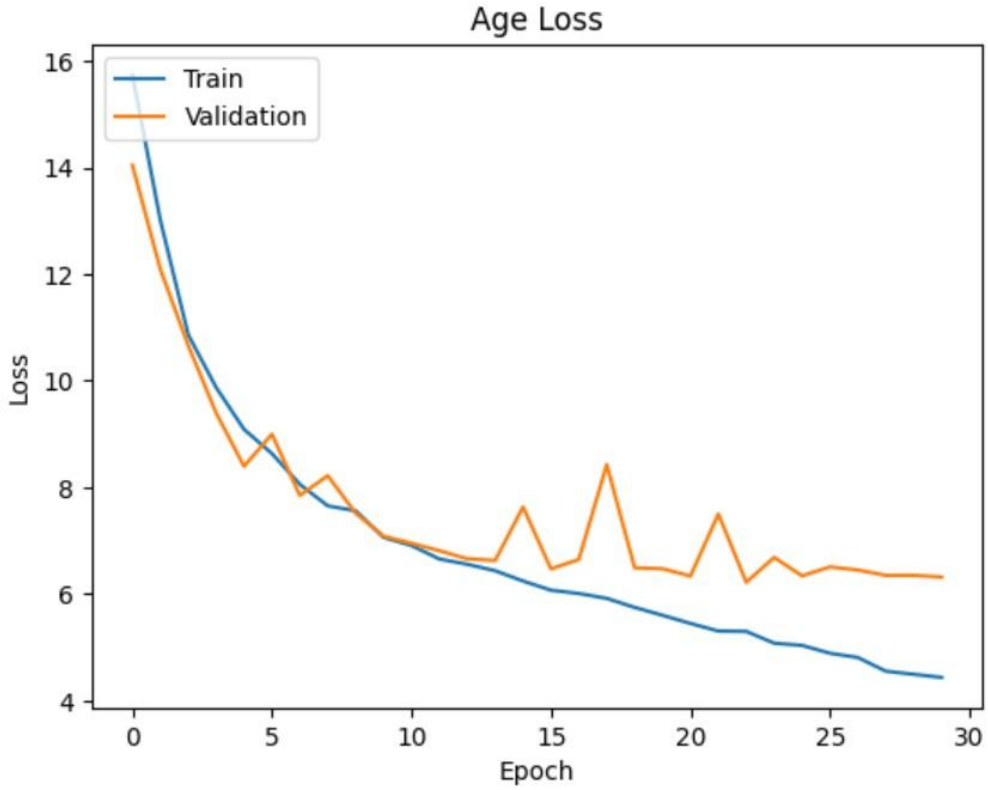
5.6 Results

Over thirty epochs the gender accuracy climbed and then levelled off, while the age error steadily decreased. The best weights were saved as best_model.keras for use in the application. A spot check on a sample image confirmed the model was producing sensible predictions by comparing its output against the true age and gender taken from the filename.

[Training vs validation accuracy curve for gender (gender_out_accuracy)]



[INSERT FIGURE: Age error curve (age_out_mae) for train and validation across epochs]



6. Task 2 — Hair Length Model (ResNet18)

6.1 First Attempt: Rule-Based Labels from CelebA

The first idea for getting hair length labels was to avoid manual work entirely by using the CelebA dataset, which comes with attribute flags such as Bangs, Wavy_Hair, Straight_Hair, Bald and Male. The plan was to combine these flags with simple rules to infer hair length automatically. For example, bald implies short hair, bangs imply long hair, and a female with wavy hair was treated as long.

```
def get_confident_label(row):
    if row['Bangs'] == 1:                  return 'long'     if
row['Male'] == 0 and row['Wavy_Hair']==1: return 'long'     if
row['Bald'] == 1:                     return 'short'     if
row['Male']==1 and row['Bald']==0 and \
       row['Bangs']==0 and row['Wavy_Hair']==0: return 'short'
return 'unknown'
```

This worked for most cases but had a serious blind spot. The CelebA attributes describe hair texture, not length, so they cannot tell the difference between a man with a short straight haircut and a man with long straight hair down to his shoulders. Both end up labelled short. The result was that the model trained on these labels learned a false rule: that straight-haired men are always short-haired.

6.2 Why This Was Abandoned

The flaw showed up dramatically in testing. A photo of a man with clearly long, straight hair was classified as short hair with 99.93 percent confidence. The problem was not the model architecture but the training data itself: it had been taught the wrong association with high confidence. Because no amount of tuning can fix bad labels, the attribute-based approach was dropped entirely in favour of manual labelling on UTKFace, as described in section 4.2.

6.3 Expanding the Dataset with Confident Auto-Labels

The 405 manual labels covered only the 20 to 30 range. To give the hair model more data to learn from, images outside that range were auto-labelled using a safe heuristic. For people under 20 or over 30, gender is a far more reliable indicator of hair length in this dataset, so females were labelled long and males short. These auto-labels were combined with the hand-made labels, but the manual labels remained the trusted ground truth for the difficult middle range.

```
for f in os.listdir(UTK_DIR):
    age, gender = int(f.split('_')[0]), int(f.split('_')[1])
    if age < 20 or age > 30:                  # only auto-label
outside 20-30        label = 'long' if gender == 1 else 'short'
        auto_labels.append({'filename': f, 'hair_length': label})
```

6.4 Why ResNet18 and Transfer Learning

With only a few hundred labelled images, training a deep network from scratch would overfit almost immediately. The solution was transfer learning: starting from a ResNet18 model that had already been trained on the large ImageNet dataset. This pretrained network already knows how to recognise general visual features like edges, textures and shapes, so it only needs to be nudged toward the specific task of judging hair length.

ResNet18 is an eighteen-layer residual network. Its key innovation is the residual or skip connection, where the input to a block of layers is added directly to that block's output. This solves a problem that affects very deep networks, where the training signal weakens as it passes back through many layers. The skip connections give the signal a shortcut, making deep networks trainable and stable. To adapt it for this task, only the final classification layer was replaced with a new one that outputs two classes, long and short:

```
from torchvision.models import resnet18, ResNet18_Weights model
= resnet18(weights=ResNet18_Weights.IMAGENET1K_V1) model.fc =
nn.Linear(model.fc.in_features, 2)   # long vs short
```

6.5 Heavy Data Augmentation

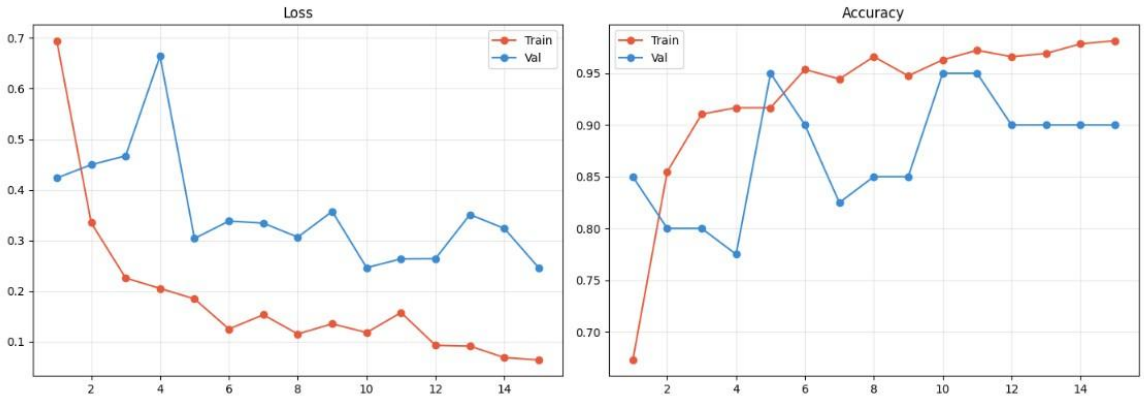
To make the small dataset behave like a larger one, strong augmentation was applied during training. Every time an image was loaded it was randomly altered, so the model effectively never saw the exact same picture twice. The augmentations included horizontal flips, colour jitter on brightness and contrast, small rotations, occasional blur, and random crops. This is one of the most effective ways to fight overfitting when labelled data is limited.

```
train_transform = transforms.Compose([
transforms.Resize((160,160)),
transforms.RandomHorizontalFlip(),    transforms.ColorJitter(0.3,
0.3, 0.3),    transforms.RandomRotation(10),
    transforms.RandomApply([transforms.GaussianBlur(3)], p=0.2),
transforms.RandomResizedCrop(128, scale=(0.8, 1.0)),    transforms.ToTensor(),
    transforms.Normalize([0.5]*3, [0.5]*3) ])
```

6.6 Training and Results

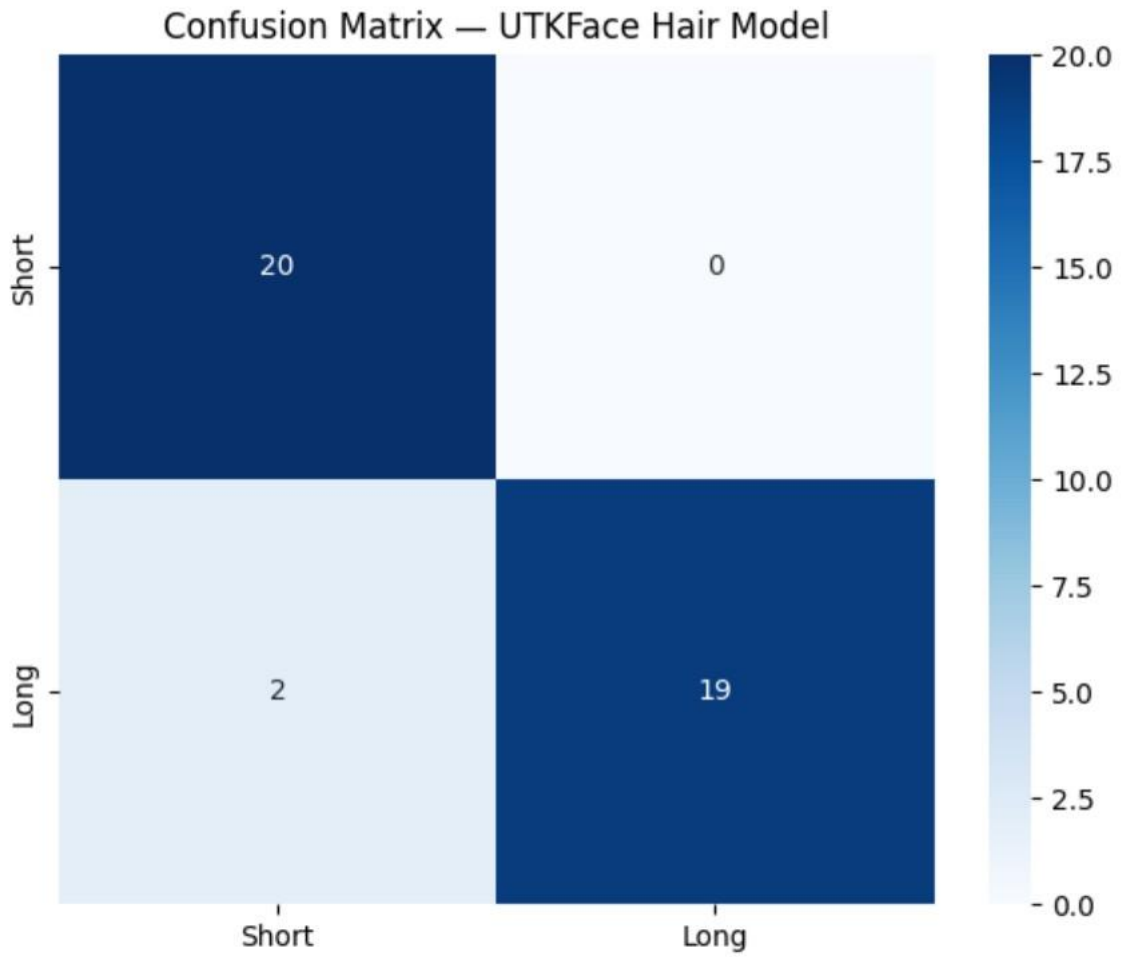
The model was trained for fifteen epochs using the Adam optimiser at a learning rate of 0.0001, with crossentropy loss and a step scheduler that halved the learning rate every five epochs. After training, the model was evaluated on the held-out test set, producing a classification report with precision, recall and F1 score for each class, along with a confusion matrix showing exactly where the predictions were correct or mistaken.

[Training and validation loss/accuracy curves (the 2-panel plot)]



	precision	recall	f1-score	support
Short hair	0.91	1.00	0.95	20
Long hair	1.00	0.90	0.95	21
accuracy			0.95	41
macro avg	0.95	0.95	0.95	41
weighted avg	0.96	0.95	0.95	41

[Confusion matrix heatmap for the hair model on the test set]



7. The Combined Decision Logic

This is the part that actually answers the problem statement. The two models supply three values — age, real gender, and hair length — and a small rule combines them. The rule reproduces exactly what the task asks for: inside the 20 to 30 band, hair length decides the gender label; outside it, the real predicted gender is used.

```
def final_gender(age, real_gender, hair_length):
if 20 <= age <= 30:
        # hair length overrides true gender in this band
return 'Female' if hair_length == 'long' else 'Male'    else:
        # outside the band, report the real predicted gender
return real_gender
```

It is worth walking through why each branch exists. The age check decides which world we are in. If the person is between 20 and 30, the task demands that appearance (hair length) wins over biology, so a long-haired man is reported as female and a short-haired woman is reported as male. If the person is any other age, the task demands the opposite — hair is ignored and the true gender is reported. The two models are only there to supply the inputs; this short function is where the actual requirement is satisfied.

Designing the system this way also makes it robust and easy to reason about. Each model does one clear job, and the combining rule is simple enough to verify by hand, which matters because the problem statement places as much weight on correct logic as on accuracy.

8. The Streamlit GUI

The problem statement requires a graphical interface, which was built with Streamlit. Streamlit was chosen because it turns a Python script into a clean web app with very little extra code, letting the focus stay on the models rather than on web development. The interface lets a user upload any face photo and instantly see the age, the hair length, and the final gender decision produced by the combined logic.

8.1 How the App Processes an Image

When an image is uploaded, the app prepares two different versions of it because the two models expect different inputs. The age and gender model needs a greyscale 128x128 image, while the hair model needs an RGB 128x128 image normalised the same way it was during training. Both versions are produced from the single uploaded photo, each model runs once, and their outputs are passed into the decision function from section 7.

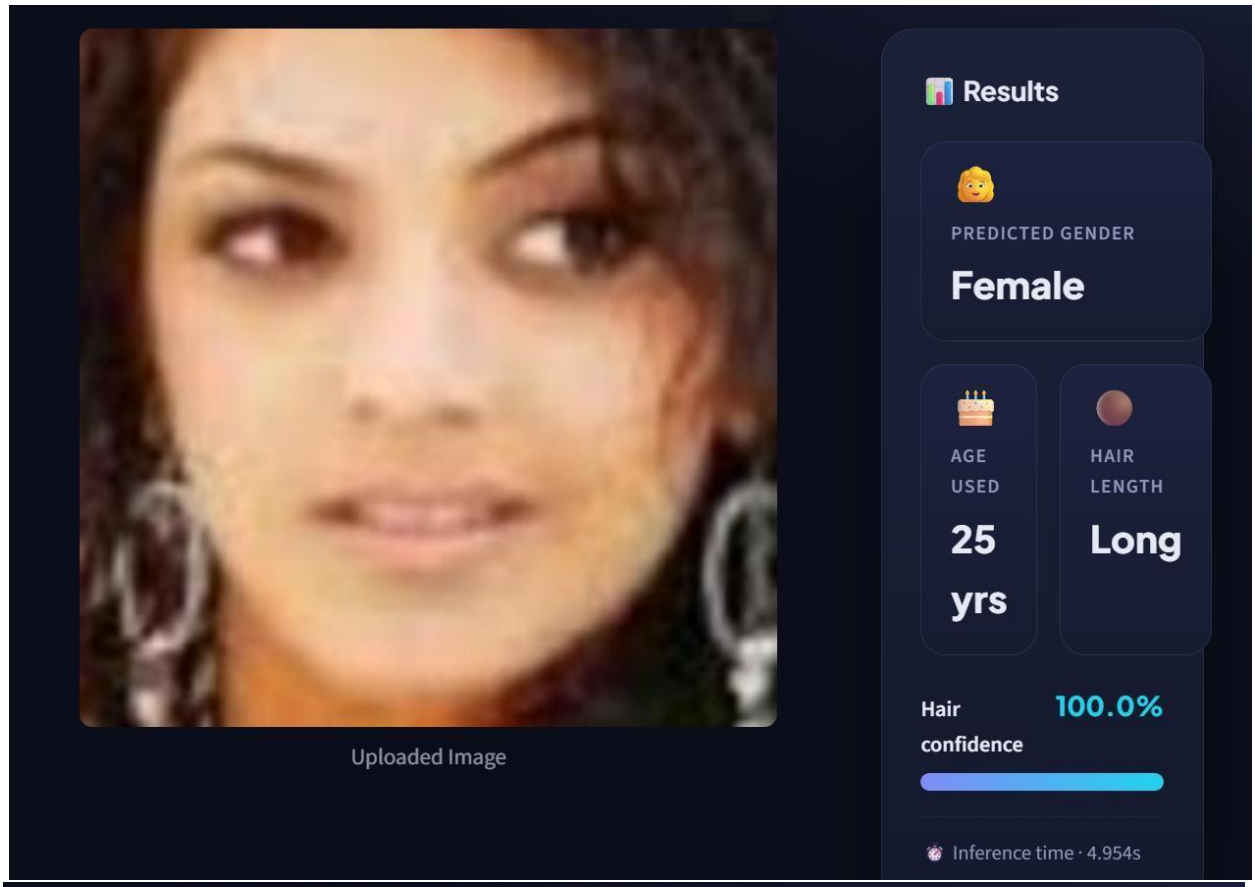
```
# 1. Age + gender  (Keras)
g_input = preprocess_grey(image)          # greyscale 128x128
gender_p, age_p = keras_model.predict(g_input)
real_gender = 'Female' if round(gender_p[0][0]) == 1 else 'Male' age
= round(age_p[0][0])

# 2. Hair length  (PyTorch ResNet18)
h_input = preprocess_rgb(image)           # RGB 128x128 tensor
logits  = hair_model(h_input)
hair    = 'long' if logits.argmax() == 1 else 'short'

# 3. Combine using the task rule result = final_gender(age,
real_gender, hair)
```

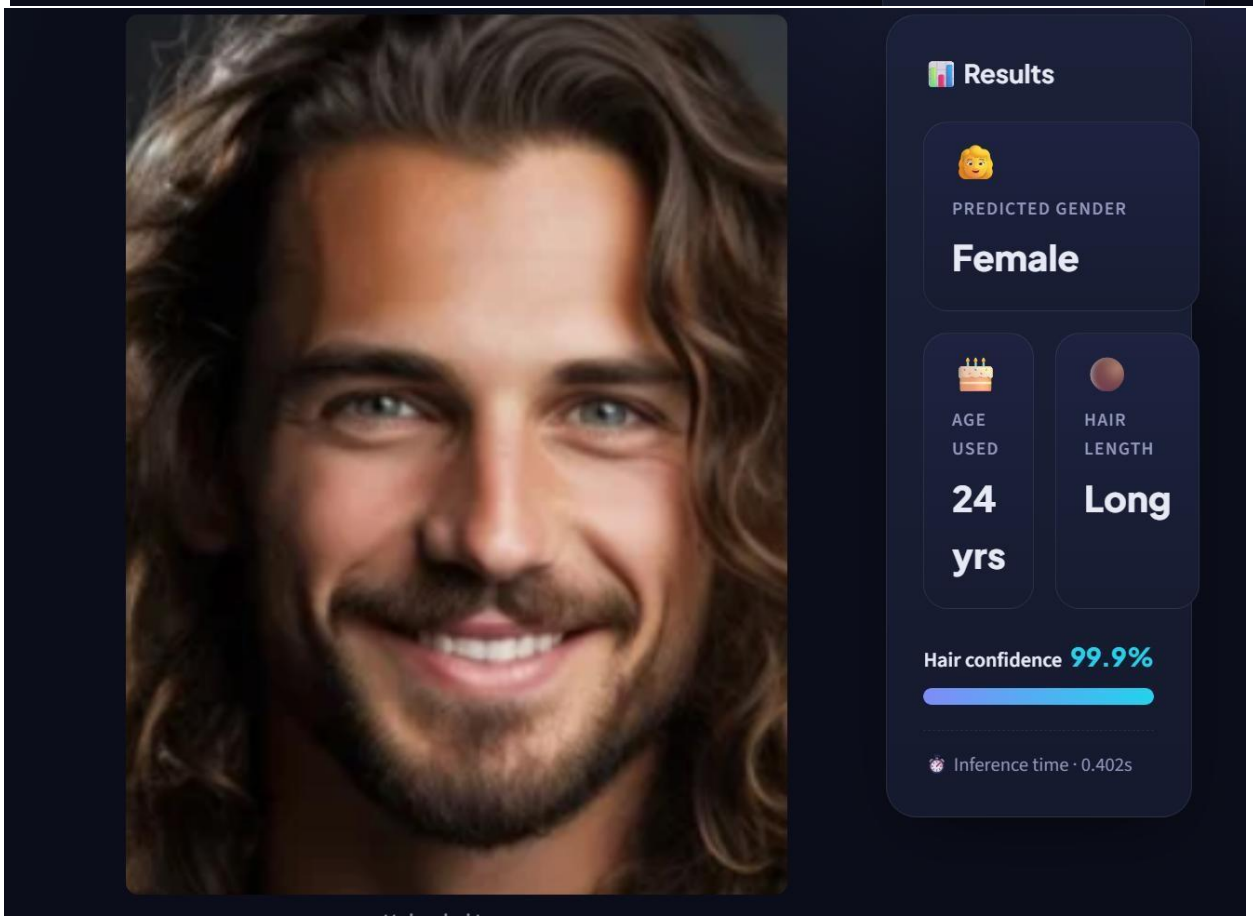
Loading two models from two different frameworks in one app is handled by loading each only once when the app starts and caching it, so that uploading a new image does not reload the models every time. This keeps the app responsive, with each prediction completing in well under a second.

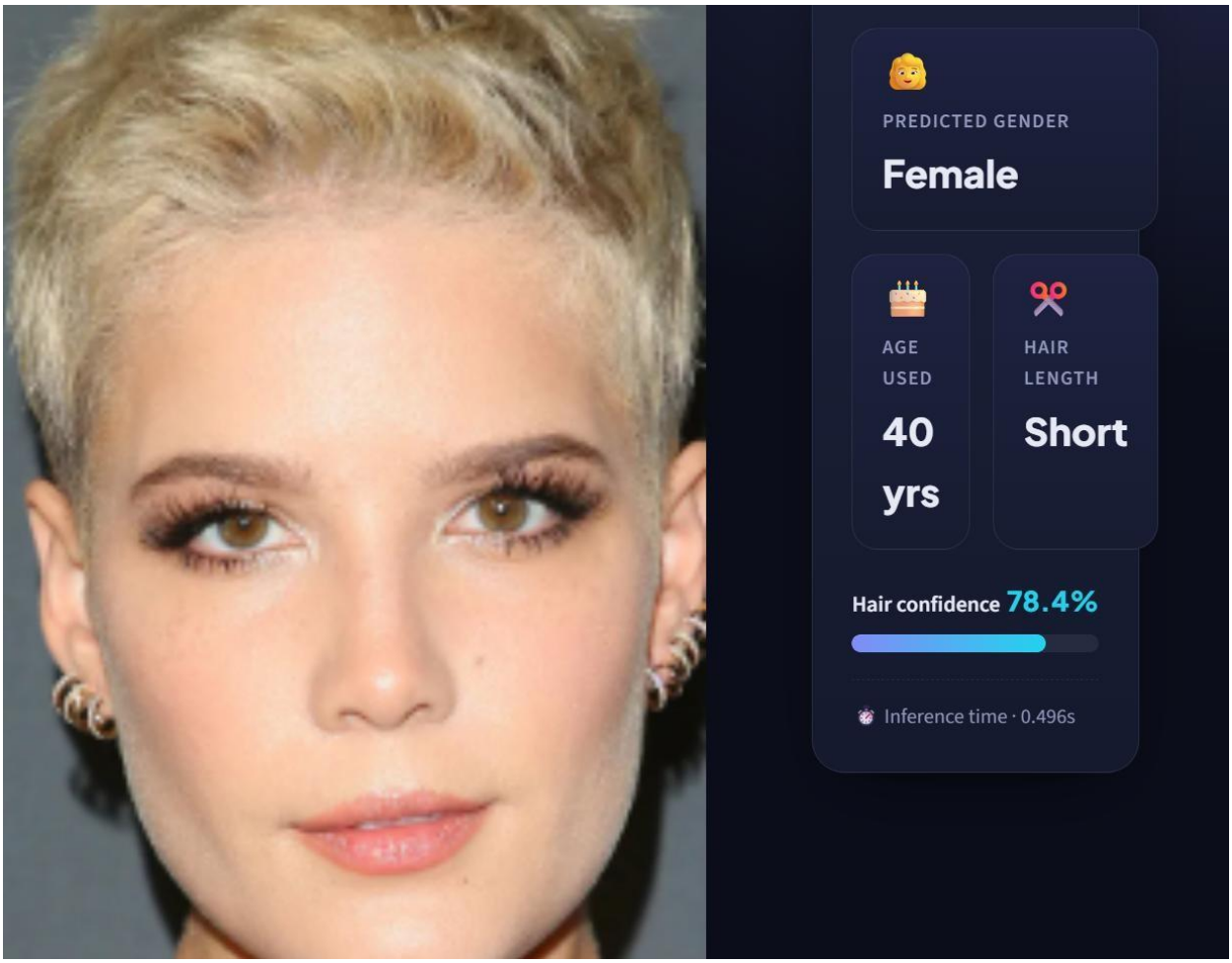
8.2 What the User Sees

The layout is intentionally simple. The uploaded image is shown on one side, and a results panel on the other displays the predicted gender, the estimated age in years, and the detected hair length together with a confidence percentage taken from the hair model's softmax output. The confidence figure gives the user a sense of how sure the model is about the hair classification, which is the value that drives the final decision inside the 20 to 30 band.

[the running Streamlit app on a sample face, showing gender, age and hair length results]


Uploaded Image
Results
PREDICTED GENDER
Female
AGE USED
25 yrs
HAIR LENGTH
Long
Hair confidence
100.0%
Inference time · 4.954s


Results
PREDICTED GENDER
Female
AGE USED
24 yrs
HAIR LENGTH
Long
Hair confidence
99.9%
Inference time · 0.402s



9. Challenges Faced

Bad hair labels from CelebA

The biggest challenge was that the first labelling strategy produced confidently wrong labels for long-haired men, because CelebA's attributes describe texture, not length. This was solved by switching to manual labelling on UTKFace, accepting more effort in exchange for trustworthy data.

A very small labelled dataset

The hair model was trained on a small custom dataset of 405 images that I built and labeled myself, by swiping through faces one at a time in an application and tagging each as short or long hair. This was extremely manual, and scaling it to the thousands of images a reliable classifier would need was simply not feasible for me to do by hand within the project timeframe. Because of that, the dataset stays small and imbalanced — it underrepresents cases like long-haired men and short-haired women — so the model leans on shortcuts (e.g. "long hair = female") and produces inconsistent results on those edge cases. I'm recording this as a known limitation: the fix is a larger, balanced dataset, but hand-collecting it at that scale was beyond what I could realistically do here.

Combining two different frameworks

The age and gender model lives in TensorFlow while the hair model lives in PyTorch. Running both in one app means loading two heavy runtimes side by side. Caching each model at startup kept the app fast and stable.

Keeping the preprocessing pipelines separate

The two models need different image formats, greyscale versus normalised RGB. Mixing these up silently produces wrong predictions, so the two preprocessing steps were kept completely separate with explicit conversions for each model.

10. Results Summary

The table below summarises the two models and the overall system. The figures are read directly from the training curves and the hair model's classification report shown earlier in this report.

Model	Task	Metric / Result
Custom CNN (Keras)	Age + Gender	Gender accuracy ≈ 90% (validation; ≈ 94% on training) | Age MAE ≈ 6.3 years (validation)
ResNet18 (PyTorch)	Hair length	Test accuracy ≈ 95%, weighted F1 ≈ 0.95 (39 of 41 test faces correct)
Combined system	Final gender per rule	Correct behaviour verified through the Streamlit GUI

For the age and gender network, the gender accuracy curve climbs quickly and then levels off near 0.90 on the validation set, while the age error settles at roughly six years of mean absolute error — meaning a typical prediction lands within about six years of the true age. The hair model reaches 0.95 accuracy on its held-out test set, with the confusion matrix showing every short-hair example correct and only two long-hair examples missed.

11. Conclusion and Future Work

This project shows that a deceptively simple-sounding requirement — identify gender, but flip it based on hair length for a specific age range — is actually a small system-design problem. The key insight was recognising that it needed three predictions, not one, and that the real work was in combining them correctly rather than in any single model.

Two models were built to supply those predictions: a custom convolutional network for age and gender, and a fine-tuned ResNet18 for hair length. A short, readable decision function ties them together to meet the exact rule in the brief, and a Streamlit interface makes the whole thing usable by anyone with a photo.

The project also produced a useful lesson about data quality. The first hair-labelling approach failed not because of the model but because of biased labels, a reminder that what a model learns is only ever as good as the data it is shown. Future improvements would include enlarging the manually labelled hair dataset to reduce reliance on heuristic auto-labels, trying more modern backbones such as EfficientNet, and adding an automatic face-detection step so the app can handle full photographs rather than pre-cropped faces.

Task 2 — Senior Citizen Identification System

Real-Time Multi-Person Age, Gender and Visit Logging for Retail Spaces

Built on the Task 1 Age + Gender model (best_model.keras), with OpenCV and a Streamlit GUI

1. Problem Statement

The second task moves from a single cropped face to a live retail environment. The system has to watch a video stream or a real-time webcam feed from a mall or local store, find every person in view, and estimate each one's age and gender. Anyone whose estimated age is 60 or above is flagged as a senior citizen. For every logged person, the system records their age, gender and the time of their visit into a CSV file, so the store ends up with a running log of who walked in and when. A graphical interface is optional for this task, but a full one was built anyway; the emphasis is on a working model and correct, reliable logging.

2. Understanding the Problem

Where Task 1 received one neatly cropped face and answered a question about it, Task 2 has to do three new things on top of the age and gender prediction it already knows how to do. First, it must **locate** faces, because a camera frame in a store contains a whole scene, not a tidy headshot — and that scene may hold several people at once. Second, it must apply a single, simple **rule** to each face: is the estimated age 60 or above? Third, it has to **persist** its findings to disk in a structured file, so the output is no longer something flashed on a screen and forgotten but a permanent, timestamped record.

Reading the task this way makes clear that the age and gender prediction is the part that is already solved. The Task 1 model, `best_model.keras`, was trained on UTKFace, which contains faces across the full age range including people well over 60, so it can be reused unchanged. The genuinely new engineering is everything around it: detecting multiple faces per frame, steadying the predictions, deciding the senior flag, avoiding logging the same person on every frame, and writing clean rows to a file.

This also closes a loop left open in Task 1, whose own future-work section noted that an automatic face-detection step would let the app handle full photographs rather than pre-cropped faces. Task 2 is essentially that step, taken all the way to a live, multi-person stream with a dashboard on top.

3. System Overview

The system is a short pipeline wrapped around the reused model and presented through a Streamlit dashboard. A frame from the camera goes first to the face detector, which returns a box for each face; every box is cropped and sent through the age and gender model; a light tracker links boxes across frames so the same person keeps one identity; a short buffer smooths each person's predictions over several frames; the senior rule is applied; and the first time a person is confirmed, their age, gender and visit time are appended to the log file. The annotated frame, live statistics and the growing log are all shown in the browser.

Component	Framework	What it produces

Age + Gender model	TensorFlow / Keras — best_model.keras, reused from Task 1	Estimated age (number) and gender (male/female) for one face
Face detector	OpenCV (Haar cascade)	A bounding box for every face found in a frame
Tracker + smoothing buffer	Plain Python	A stable ID per person and a steadied age/gender over 10 frames
Senior + logging logic	Plain Python	Applies the age-≥-60 rule and writes a timestamped row to the CSV
Graphical interface	Streamlit	Webcam/upload modes, live stats, and the visit-log viewer with CSV download

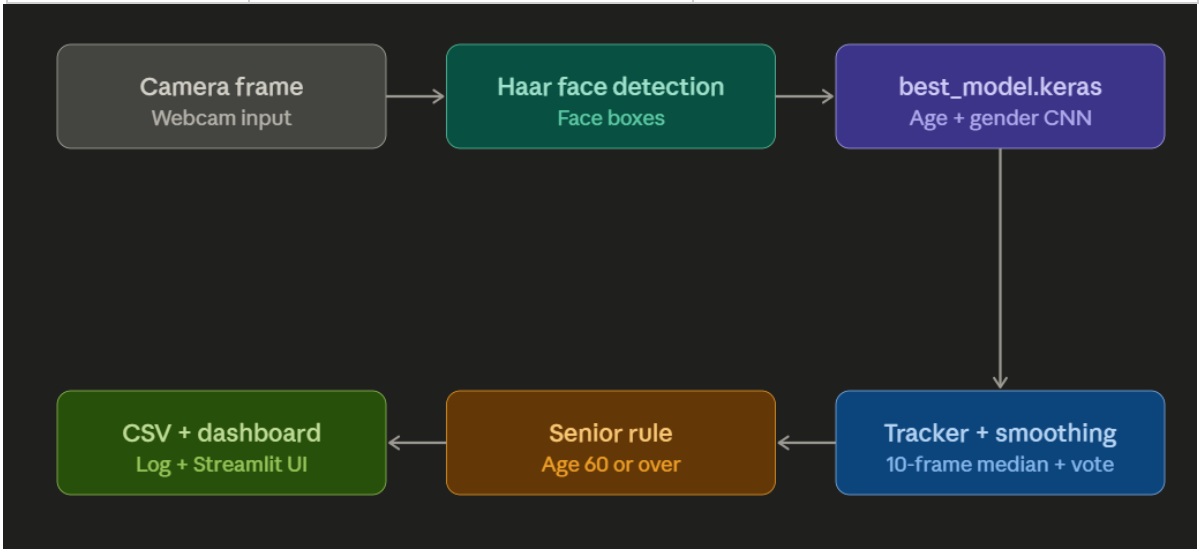


Figure 1 — pipeline block diagram: camera frame → Haar face detection → best_model.keras (age + gender) → tracker + 10-frame smoothing → senior rule → CSV + Streamlit dashboard.

4. Reusing the Age + Gender Model (best_model.keras)

The single most useful decision here was not to train anything new. The model saved at the end of Task 1 already predicts gender and a continuous age from a face, so it is loaded exactly as it was saved — and cached with Streamlit's `@st.cache_resource` so it is read from disk only once rather than on every frame. The one firm requirement is that each detected face is preprocessed in the identical way it was during training, otherwise the model would be fed inputs it never saw.

```
@st.cache_resource                        # load once, not on every frame / rerun
def load_model():
    return tf.keras.models.load_model("best_model.keras")   # reused from Task 1

model = load_model()
```

```
def predict(face_bgr):
    # identical preprocessing to Task 1: greyscale -> 128x128 -> /255 ->
(1,128,128,1)
    gray = cv2.cvtColor(face_bgr, cv2.COLOR_BGR2GRAY)
    gray = cv2.resize(gray, (128, 128))
    inp  = (gray / 255.0).reshape(1, 128, 128, 1)

    gender_pred, age_pred = model.predict(inp, verbose=0)   # same output order as
Task 1
    gender = "Female" if float(gender_pred[0][0]) > 0.5 else "Male"


    age    = int(age_pred[0][0])
    return age, gender
```

Keeping the preprocessing byte-for-byte identical to Task 1 is what lets the existing model work on new, detector-cropped faces without any retraining. Gender comes back as a probability that is thresholded at 0.5, and age as a raw number rounded to a whole year.

5. Detecting Multiple People in a Frame

The reused model expects a single face, but a store camera shows a scene. A face detector bridges that gap by scanning each frame and returning a rectangle around every face it finds. OpenCV's Haar cascade detector was used because it ships with the library, needs no extra download, and runs fast enough for a live feed on an ordinary laptop. Its `detectMultiScale` call returns a list of boxes — one per face — which is exactly the multiple-person capability the task needs: the rest of the pipeline simply loops over that list.

```
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml")

def detect_faces(frame):
    gray  = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)     faces =
face_cascade.detectMultiScale(gray, scaleFactor=1.1,
minNeighbors=5, minSize=(60, 60))     return [(x, y, x + w, y + h) for (x, y, w,
h) in faces]   # one box per face
```

For crowded scenes or faces seen at an angle, a deep-learning detector such as OpenCV's res10 SSD model or MTCNN would catch more faces and is a natural upgrade. The Haar cascade was chosen here for simplicity and zero setup, and because the detector only has to hand back boxes, it can be swapped without touching the model or the logging code.

6. The Senior-Citizen Decision Logic

The rule the task asks for is deliberately small: a person is a senior citizen if their estimated age is 60 or above. It is applied to every detected face independently, and — unlike Task 1, where hair length could override the true gender — it does not change the gender prediction at all. Gender here is always the model's real prediction. The threshold lives in one constant, so it is trivial to change to a strict "over 60" boundary if a marker prefers that reading of the brief.

```
SENIOR_THRESHOLD = 60                # a single constant, easy to adjust

def is_senior(age):
    return age >= SENIOR_THRESHOLD     # 60 or older is flagged
```

7. Tracking and Temporal Smoothing

Two problems appear the moment the system runs on video rather than a single image: the same person reappears in dozens of consecutive frames, and a single frame's prediction can be noisy.
Both are handled together. A lightweight tracker assigns each face a stable ID by matching detections between frames on how close their centres are, so one visitor keeps one identity.

```
def get_face_id(x1, y1, x2, y2, existing_ids, tolerance=60):
"""Match a detected box to an existing tracked face by how close
its centre is; return a matched ID or create a new one."""     cx, cy
= (x1 + x2) // 2, (y1 + y2) // 2     for fid, data in
existing_ids.items():
        fx, fy = data["center"]         if abs(cx - fx) <
tolerance and abs(cy - fy) < tolerance:
            existing_ids[fid]["center"] = (cx, cy)
return fid     new_id = len(existing_ids)
existing_ids[new_id] = {"center": (cx, cy)}
return new_id
```

Each tracked face then keeps a short rolling buffer of its last ten predictions. The displayed age is the *median* of those readings, which shrugs off the occasional wild frame, and the displayed gender is a *majority vote*. Until the buffer fills, the label simply reads "Analysing…", and the visitor is written to the log only once — the moment the buffer is full and the result is trusted — not on every frame.

```
BUFFER_SIZE = 10                         # collect 10 frames before trusting a
result

buf["ages"].append(age_raw)              # raw per-frame predictions for this face
buf["genders"].append(gender_raw)

stable_age    = int(np.median(buf["ages"]))                          # robust to
outliers
stable_gender = max(set(buf["genders"]), key=buf["genders"].count)  # majority
vote

# log this visitor only once, after the buffer has filled
if len(buf["ages"]) == BUFFER_SIZE and not buf["logged"]:
    if is_senior(stable_age) or log_all:
        log_record(stable_age, stable_gender)
    buf["logged"] = True
```

This is the practical answer to the 60-year boundary worry from Task 1: because the age output carries a few years of error, a single frame can flip a borderline person across the threshold, but the median over ten frames is far steadier and gives a much more reliable senior decision.

8. Logging Visits to CSV

The persistent output of the whole task is the file. On startup the system creates `senior_visit_log.csv` with a header row if it does not already exist, and from then on each confirmed visitor appends a single line holding the time of visit, the estimated age, the gender, and whether they were flagged as a senior citizen.

```
CSV_FILE = "senior_visit_log.csv"

def log_record(age, gender):
    record = {
        "Timestamp":      datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "Age":            age,
        "Gender":         gender,
        "Senior Citizen": "Yes" if age >= SENIOR_THRESHOLD else "No",       }
pd.DataFrame([record]).to_csv(CSV_FILE,  mode="a",  header=False,  index=False)
return record
```

By default the log records seniors only, which keeps the file focused on the people the task cares about; a "log all persons" switch in the sidebar flips this on to record every detected visitor instead. Each row is self-contained and timestamped, so the file doubles as a simple footfall record the store can open in Excel or analyse later. A few example rows look like this:

Timestamp	Age	Gender	Senior Citizen
2026-06-23 14:02:11	67	Male	Yes
2026-06-23 14:03:48	62	Female	Yes
2026-06-23 14:05:20	71	Male	Yes

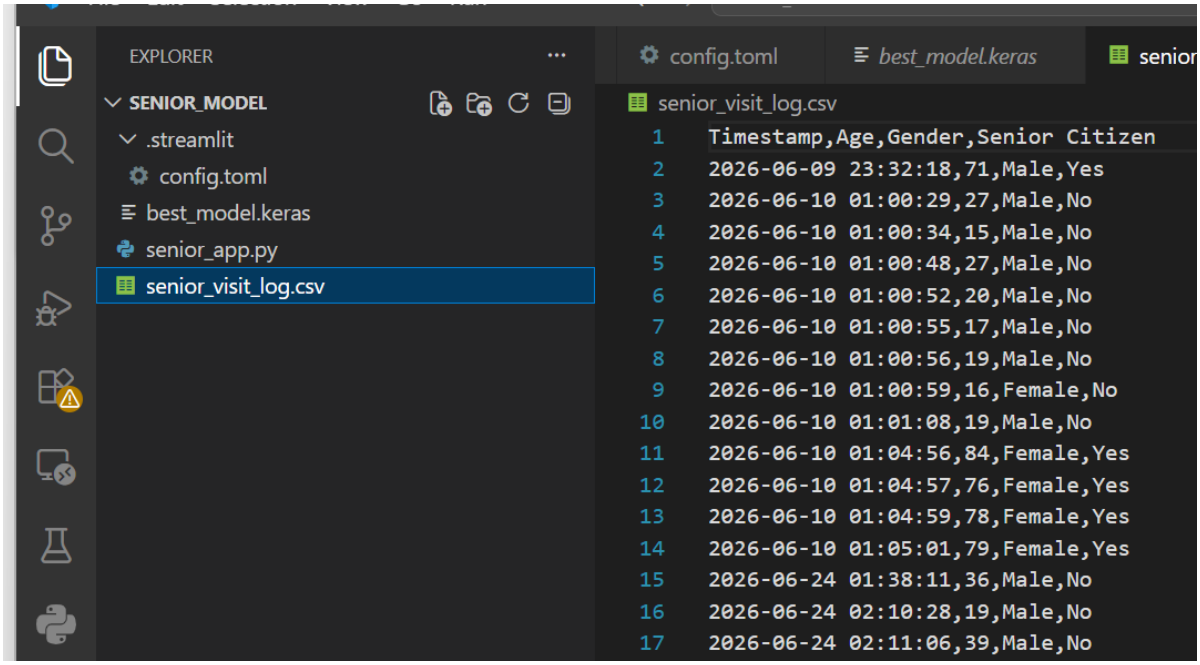


Figure 2 — the generated senior_visit_log.csv opened in a spreadsheet (or the in-app log table), showing real logged rows from a run.

9. The Graphical Interface (Streamlit)

Although the task makes a GUI optional, the system was given a full Streamlit dashboard so it can be run and demonstrated without touching the command line. Streamlit was chosen for the same reason as in Task 1 — it turns a plain Python script into a web app with very little extra code — and the app is styled with a dark theme so the red senior highlights stand out clearly against the video.

The interface offers two input modes, selected with a single toggle at the top:

Live webcam

A "Start Webcam" switch opens the camera and runs the detect → predict → smooth → log loop on the incoming frames, drawing boxes and labels straight onto the video shown in the browser. To keep it responsive, a "process every N frames" slider lets the user skip frames so only every Nth one is analysed.

```
run = st.toggle("Start Webcam")
if run:
    cap = cv2.VideoCapture(0)            # 0 = webcam
```

```
    frame_count = 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        frame_count += 1
        if frame_count % frame_skip == 0:                    # skip frames for speed
            boxes              = detect_faces(frame)
            annotated, detects = annotate(frame.copy(), boxes, log_all)  #
predict+smooth+log
            for age, gender in detects:
                log_record(age, gender)
            frame_box.image(cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB))  # live
view
    cap.release()
```

[paste your screenshot / figure here]

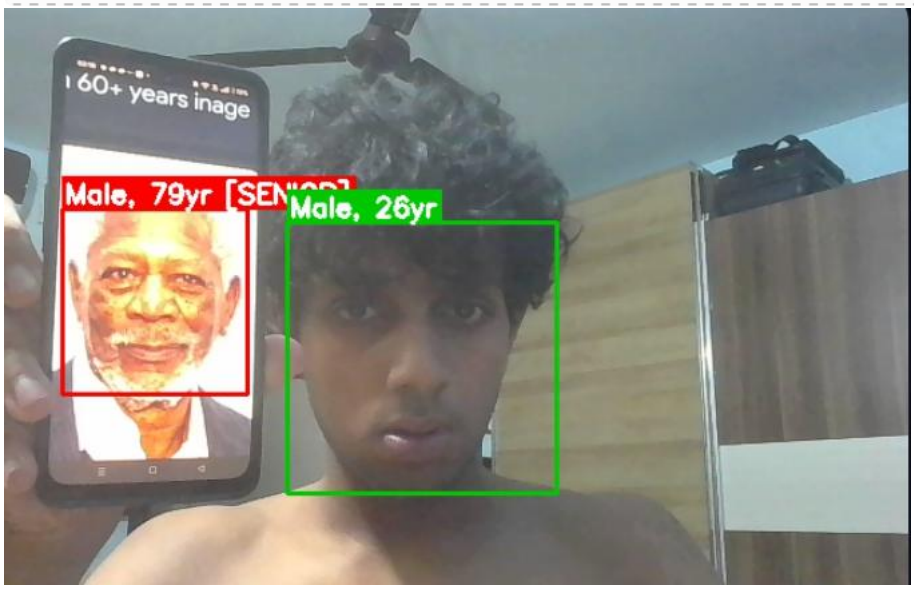


Figure 3 — live webcam mode running on a multi-person scene, with senior citizens (60+) boxed in red and everyone else in green, each face labelled with gender and age.

Upload image

A single still photo is analysed in one shot — without temporal smoothing, since there is only one frame — and every detected face is boxed, labelled and listed in a small table.

ure 4 — upload-image mode: an uploaded photo with every detected face boxed and labelled, alongside the perface results table.

Alongside the video or image, a stats panel shows live session counts as coloured cards: the total number of people logged, how many were seniors aged 60+, and the male/female split, with a table of the most recent detections underneath. A sidebar holds the controls — the "log all persons" switch (off by default), the frame-skip slider, and a "clear log" button — plus a small panel restating the model, detector, senior rule and smoothing settings. At the bottom of the page a full visit-log viewer reads `senior_visit_log.csv` back, shows the running totals and the complete table, and offers a one-click CSV download, so the same file the detection loop writes is immediately viewable and exportable from the GUI.

Timestamp	Age	Gender	Senior Citizen
2026-06-10 01:00:48	27	Male	No
2026-06-10 01:00:52	20	Male	No
2026-06-10 01:00:55	17	Male	No
2026-06-10 01:00:56	19	Male	No
2026-06-10 01:00:59	16	Female	No
2026-06-10 01:01:08	19	Male	No
2026-06-10 01:04:56	84	Female	Yes
2026-06-10 01:04:57	76	Female	Yes
2026-06-10 01:04:59	78	Female	Yes
2026-06-10 01:05:01	79	Female	Yes

Figure 5

10. Challenges Faced

Reusing a model trained on clean crops

The trained model works reasonably well for age prediction, but its performance depends on certain conditions. It requires good lighting and a clear view of the face for accurate analysis. The prediction process can sometimes take a few seconds, but it generally estimates the correct approximate age. The person also needs to be relatively close to the camera so that the face is captured clearly and can be properly analyzed. Under poor lighting, extreme angles, or when the face is too far from the camera, the accuracy of the predictions may decrease.

Steadying noisy predictions at the 60 boundary

Because the age output is a regression with a few years of error, a single frame can push a borderline person over or under 60. Taking the median age and a majority-vote gender over a tenframe buffer made the live result and the senior decision noticeably more stable.

Avoiding duplicate log rows

Video repeats each person across many frames, so a proximity tracker plus a "log once per face appearance" flag was needed to stop the CSV filling up with duplicates of the same visitor.

Missed faces in crowds

Haar cascades miss profile and partially hidden faces in busy scenes, so some visitors go uncounted. A deep-learning face detector is the clear upgrade path and slots in without changing the model or the logging.

11. Results Summary

Part	Role	Result
best_model.keras (reused)	Age + gender per face	Same model as Task 1, no retraining required
OpenCV Haar detector	Find every face per frame	Enables true multi-person detection
Tracker + 10-frame buffer	One stable, de-duplicated entry per visitor	Median age + majority-vote gender, logged once
Senior rule + CSV logging	Flag age ≥ 60, write the file	senior_visit_log.csv with a timestamp per visitor
Streamlit dashboard	Webcam / upload UI, live stats, CSV download	Full optional GUI delivered

The system runs on a live webcam or an uploaded image, detects several people at once, smooths each person's age and gender over time, marks anyone estimated at 60 or above as a senior citizen, and produces `senior_visit_log.csv` holding the age, gender and visit time of each logged person — meeting the task's required behaviour, with a styled Streamlit dashboard provided as the optional interface.

12. Conclusion and Future Work

Task 2 reuses the Task 1 age and gender model unchanged and wraps it in the machinery a real deployment needs: face detection to handle whole scenes, tracking and a smoothing buffer to count each person once and steady the result, a one-line senior rule, durable logging to a file, and a Streamlit dashboard to drive and inspect all of it. The lesson that carries over from Task 1 is that most of the work lives outside the model — in detecting, smoothing, deciding and recording — and that a clear pipeline is what turns a single prediction into a usable system.

Future improvements would include swapping the Haar cascade for a deep-learning face detector to catch more people in busy scenes, extending the log with a unique visitor ID and a dwell time so the store can measure how long each person stays, and adding a privacy notice, since a system that reads age and gender from people in a public space and writes it to a file carries real consent and data-retention obligations in a genuine deployment.